Daffodil International University

DEPARTMENT OF INFORMATION & COMMUNICATION ENGINEERING

Dhaka, BANGLADESH.

COURSE NO.: ICE 3242

COURSE TITLE: DIGITAL SIGNAL PROCESSING Lab

Experiment No. 04

Name of Experiment: Sampling, Quantization and Aliasing of Signals

---

Objectives:

The objectives of this experiment are:

1. To sample a continuous-time (analog) sine wave and obtain a discrete-time signal.
2. To understand the sampling theorem, the Nyquist rate and the Nyquist frequency.
3. To observe under-sampling and over-sampling and to see how aliasing happens.
4. To quantize a signal with a given number of bits and observe the quantization error.

THEORY:

Real-world signals such as speech, music and temperature are *analog*: they exist at every instant of time and can take any value. A computer can store only a list of numbers. To convert an analog signal into a digital signal, two steps are needed:

- **Sampling** – take the value of the signal only at equally spaced instants of time (makes the time discrete).
- **Quantization** – round each sample value to one of a fixed number of levels (makes the amplitude discrete).

Both steps are done by an Analog-to-Digital Converter (ADC). The discrete-time signals generated in Experiments 01–03 can be thought of as the output of such a converter.

1. Sampling

In sampling, the analog signal $x_a(t)$ is measured every T_s seconds. T_s is called the **sampling period** and $f_s = 1/T_s$ is the **sampling frequency** (number of samples per second, in Hz). The n-th sample is

$$x(n) = x_a(nT_s) = x_a\left(\frac{n}{f_s}\right), \qquad n = 0,\ 1,\ 2,\ \ldots$$

For an analog sine wave of frequency f Hz, $x_a(t) = \sin(2\pi ft)$, the sampled signal becomes

$$x(n) = \sin\left(2\pi \frac{f}{f_s} n\right)$$

Example: a 5 Hz sine wave sampled at $f_s = 50$ Hz gives 50 samples per second, i.e. 50/5 = 10 samples in every cycle. Fig. 4.1 shows the analog signal and its samples (output of Program 1).

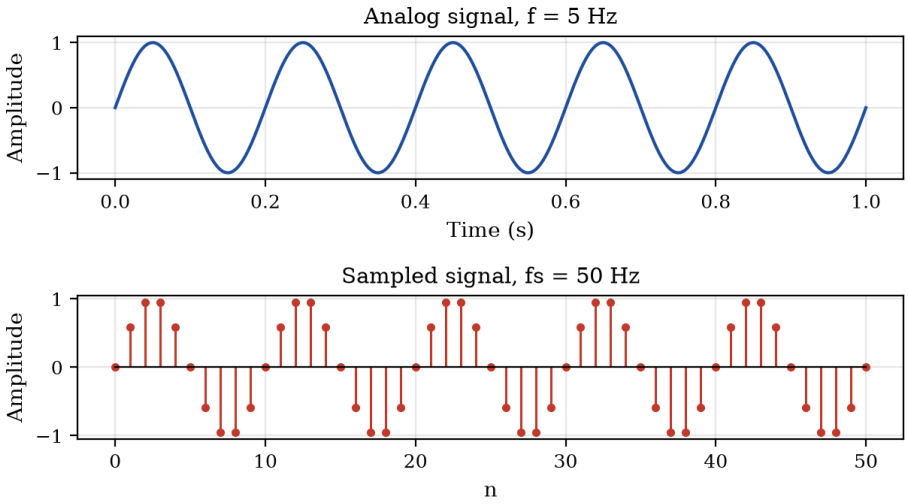


Fig. 4.1 Analog sine wave (top) and its sampled version x(n) (bottom)

2. Sampling Theorem

The sampling theorem tells us how fast we must sample so that no information is lost:

Sampling theorem: a signal whose highest frequency is f_{max} can be recovered exactly from its samples if the sampling frequency is more than twice f_{max}, i.e. $f_s > 2f_{max}$.

$$f_s > 2f_{max}$$

Two terms come from this theorem. Students often mix them up, so remember the difference:

Table 4.1 Nyquist rate and Nyquist frequency

Term	Formula	Depends on	Example (f = 5 Hz, f_s = 50 Hz)
Nyquist rate	2 f_{max}	the signal	2 × 5 = 10 Hz (minimum safe sampling rate)
Nyquist frequency	f_s / 2	the sampler	50 / 2 = 25 Hz (highest frequency that can be represented)

3. Under-sampling, Critical Sampling and Over-sampling

Comparing the sampling frequency with the Nyquist rate (2f) gives three cases. For a 5 Hz sine wave (Nyquist rate 10 Hz):

Table 4.2 The three sampling cases

Case	Condition	Example	Result
Under-sampling	$f_s < 2f$	f_s = 8 Hz	Too few samples – aliasing occurs; the signal cannot be recovered.
Critical sampling	$f_s = 2f$	f_s = 10 Hz	Only 2 samples per cycle – not safe (the samples may even all be zero).
Over-sampling	$f_s > 2f$	f_s = 12 Hz, 50 Hz	Enough samples – the signal can be recovered; more samples look more like the analog wave.

Fig. 4.2 shows the 5 Hz sine wave sampled at 8 Hz, 12 Hz and 50 Hz (output of Program 2, run three times). At 8 Hz the red samples do not follow the 5 Hz wave at all; at 50 Hz they follow it closely.

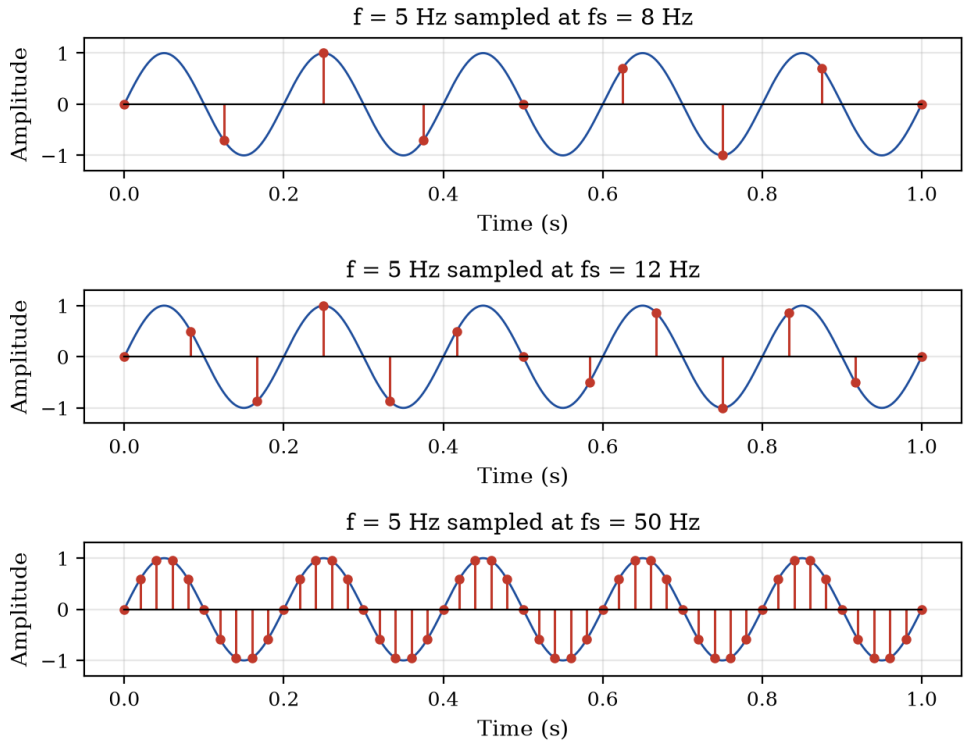


Fig. 4.2 Effect of the sampling frequency: f_s = 8 Hz (under-sampling), 12 Hz and 50 Hz (over-sampling)

4. Aliasing

When a signal is under-sampled, a high frequency *pretends* to be a lower frequency. This is called **aliasing**, and the false frequency is called the **alias**. It happens because two sine waves whose frequencies differ by f_s give exactly the same samples. The apparent (alias) frequency is

$$f_{alias} = |f - k\,f_s|, \qquad k = \text{the integer nearest to } f/f_s$$

- 9 Hz sampled at 8 Hz: k = 1, f_alias = |9 − 8| = **1 Hz**.
- 5 Hz sampled at 8 Hz: k = 1, f_alias = |5 − 8| = **3 Hz** (this is what is seen in the top plot of Fig. 4.2).
- 5 Hz sampled at 50 Hz: k = 0, f_alias = 5 Hz – no aliasing.

Fig. 4.3 (output of Program 3) shows a 9 Hz and a 1 Hz sine wave sampled at 8 Hz. The samples of both waves are identical, so after sampling the two signals cannot be told apart. In practice an **anti-aliasing low-pass filter** is placed before the ADC to remove frequencies above $f_s/2$.

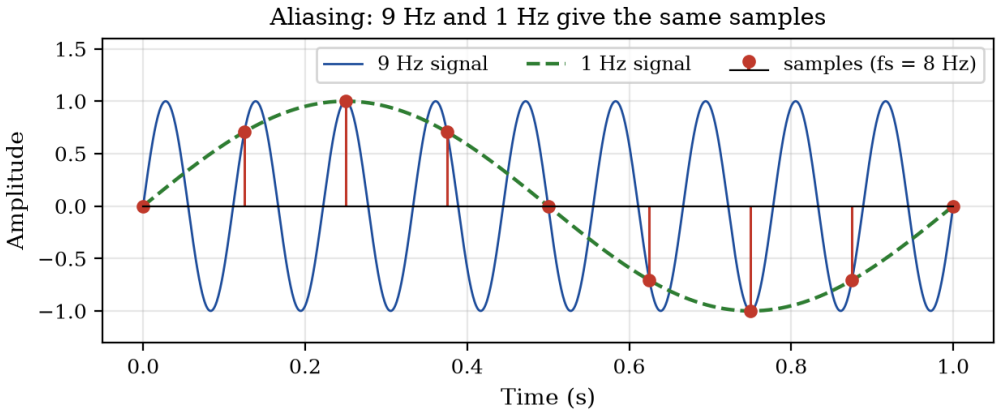


Fig. 4.3 Aliasing – a 9 Hz and a 1 Hz sine wave give identical samples at f_s = 8 Hz

5. Quantization

After sampling, each sample can still have any value. A B-bit ADC can store only $L = 2^B$ different values, called **quantization levels**. Quantization replaces every sample by the *nearest* level. For a signal between −1 and +1, the distance between two levels (the **step size** Δ) is

$$L = 2^B, \qquad \Delta = \frac{x_{max} - x_{min}}{L - 1} = \frac{2}{L - 1}$$

The difference between the quantized value and the true value is the **quantization error**. Because every sample is rounded to the nearest level, the error is never larger than half a step:

$$e(n) = x_q(n) - x(n), \qquad |e(n)| \le \frac{\Delta}{2}$$

Table 4.3 More bits give more levels and a smaller error

Bits B	Levels L = 2^B	Step size Δ = 2/(L − 1)	Maximum error Δ/2
2	4	0.6667	0.3333
3	8	0.2857	0.1429
4	16	0.1333	0.0667
8	256	0.0078	0.0039

Fig. 4.4 (output of Program 4) shows a 2 Hz sine wave quantized with B = 3 bits (8 levels). The quantized signal looks like a staircase, and the error stays between −0.143 and +0.143.

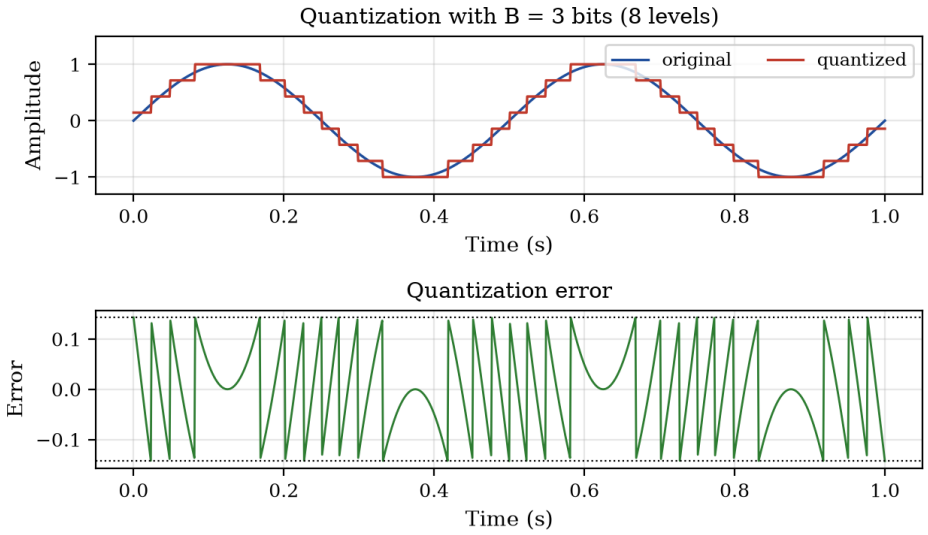


Fig. 4.4 Quantized signal (top) and quantization error, bounded by ±Δ/2 (bottom)

Important Equations

$$T_s = \frac{1}{f_s}, \qquad x(n) = x_a(nT_s) \tag{4.1}$$

$$f_s > 2f_{max} \qquad \text{(sampling theorem)} \tag{4.2}$$

$$\text{Nyquist rate} = 2f_{max}, \qquad \text{Nyquist frequency} = \frac{f_s}{2} \tag{4.3}$$

$$f_{alias} = |f - k\,f_s| \tag{4.4}$$

$$L = 2^B, \qquad \Delta = \frac{x_{max} - x_{min}}{L - 1}, \qquad |e(n)| \le \frac{\Delta}{2} \tag{4.5}$$

MATLAB commands used

- **`t = 0:0.001:1`** – creates a very fine time axis; the signal drawn on it looks continuous.
- **`sin(2*pi*f*t)`** – generates a sine wave of frequency f Hz.
- **`plot(t,x)`** – draws a continuous-looking curve (used for the analog signal).
- **`stem(n,x)`** – draws the samples of a discrete-time signal.
- **`hold on`** – keeps the old plot so that a new plot is drawn on top of it.
- **`input('...')`** – reads a value typed by the user.
- **`round(x)`** – rounds each value to the nearest integer (used for quantization).
- **`max(abs(e))`** – finds the largest absolute value in a vector.
- **`num2str(a)`** – converts a number into text (used inside titles).

PROCEDURE:

Apparatus Required

- System with MATLAB 7.0.

Algorithm

1. Generate a 5 Hz analog sine wave on a fine time axis t = 0 : 0.001 : 1.
2. Choose a sampling frequency fs and form the sample index n = 0, 1, …, fs (one second of samples).
3. Compute the samples $x(n) = \sin(2\pi fn/fs)$ and plot them with stem().
4. Repeat with fs = 8 Hz, 12 Hz and 50 Hz and compare the samples with the analog wave.
5. Sample a 9 Hz and a 1 Hz sine wave at fs = 8 Hz and check that their samples are equal (aliasing).
6. Quantize a sine wave with B bits using the step size $\Delta = 2/(2^B - 1)$ and plot the quantized signal and the error.

Program

1. Sampling of a Sine Wave

```
% 1. SAMPLING OF A SINE WAVE
clc;
clear all;
close all;
f = 5;                    % signal frequency (Hz)
t = 0:0.001:1;            % fine time axis
xa = sin(2*pi*f*t);       % analog signal
fs = 50;                  % sampling frequency (Hz)
n = 0:fs;                 % sample number
x = sin(2*pi*f*n/fs);     % sampled signal x(n)
subplot(2,1,1);
plot(t,xa);
xlabel('Time (s)');
ylabel('Amplitude');
title('Analog signal, f = 5 Hz');
subplot(2,1,2);
stem(n,x);
xlabel('n');
ylabel('Amplitude');
title('Sampled signal, fs = 50 Hz');
```

2. Under-sampling and Over-sampling

```
% 2. UNDER-SAMPLING AND OVER-SAMPLING
clc;
clear all;
close all;
f = 5;                                % signal frequency (Hz)
fs = input('Sampling frequency (Hz) = ');
t = 0:0.001:1;
xa = sin(2*pi*f*t);                   % analog signal
n = 0:fs;
x = sin(2*pi*f*n/fs);                 % samples
plot(t,xa);
hold on;
stem(n/fs,x,'r');                     % samples at time n/fs
xlabel('Time (s)');
ylabel('Amplitude');
title(['f = 5 Hz sampled at fs = ', num2str(fs), ' Hz']);
```

3. Aliasing

```
% 3. ALIASING
clc;
clear all;
close all;
fs = 8;                  % sampling frequency (Hz)
t = 0:0.001:1;
n = 0:fs;
x1 = sin(2*pi*9*n/fs);    % samples of a 9 Hz sine
x2 = sin(2*pi*1*n/fs);    % samples of a 1 Hz sine
plot(t,sin(2*pi*9*t));
hold on;
plot(t,sin(2*pi*1*t),'g');
stem(n/fs,x1,'r');
xlabel('Time (s)');
ylabel('Amplitude');
title('Aliasing: 9 Hz and 1 Hz give the same samples');
disp('Samples of 9 Hz signal:');  disp(x1);
disp('Samples of 1 Hz signal:');  disp(x2);
```

4. Quantization

```
% 4. QUANTIZATION
clc;
clear all;
close all;
B = 3;                   % number of bits
L = 2^B;                 % number of levels
t = 0:0.001:1;
x = sin(2*pi*2*t);       % 2 Hz sine wave
D = 2/(L-1);             % step size
xq = round((x+1)/D)*D - 1;   % quantized signal
e = xq - x;              % quantization error
subplot(2,1,1);
plot(t,x,t,xq,'r');
legend('original','quantized');
xlabel('Time (s)');
ylabel('Amplitude');
title('Quantization with B = 3 bits');
subplot(2,1,2);
plot(t,e);
xlabel('Time (s)');
ylabel('Error');
title('Quantization error');
disp('Step size =');      disp(D);
disp('Maximum error =');  disp(max(abs(e)));
```

Run Program 2 three times and enter fs = 8, 12 and 50 when asked.

Program Explanation

MATLAB statement	What it does	Why it is used
`clc; clear all; close all;`	Clears the screen, the variables and old figures.	Each program starts fresh.
`f = 5;`	Sets the signal frequency to 5 Hz.	This is the frequency we want to sample.
`t = 0:0.001:1;`	Makes 1001 time points, 1 ms apart, from 0 to 1 s.	Very close points make the plot look like an analog (continuous) signal.

MATLAB statement	What it does	Why it is used
`xa = sin(2*pi*f*t);`	Computes the analog sine wave.	Reference signal to compare with the samples.
`fs = 50;`	Sets the sampling frequency to 50 Hz.	50 samples are taken every second.
`n = 0:fs;`	Sample numbers 0, 1, …, fs.	Gives one second of samples (fs + 1 points).
`x = sin(2*pi*f*n/fs);`	Computes the samples x(n) = xa(n/fs).	This is sampling: time t is replaced by n/fs.
`stem(n,x);`	Draws the samples as vertical lines.	Sampled signals are discrete, so stem is used.
`fs = input('...');`	Asks the user to type the sampling frequency.	The same program can test 8, 12 and 50 Hz.
`hold on;`	Keeps the analog curve on the figure.	Samples are drawn on top of the analog signal.
`stem(n/fs,x,'r');`	Plots the samples (red) at their real times n/fs.	Shows where the samples fall on the analog wave.
`title(['...', num2str(fs), ' Hz']);`	Builds a title that contains the value of fs.	Each figure shows which fs was used.
`x1 = sin(2*pi*9*n/fs);` `x2 = sin(2*pi*1*n/fs);`	Samples a 9 Hz and a 1 Hz sine at fs = 8 Hz.	9 − 8 = 1, so both must give the same samples.
`disp(x1); disp(x2);`	Prints both sample lists.	Proves numerically that the samples are equal (aliasing).
`B = 3;  L = 2^B;`	3 bits give $2^3 = 8$ levels.	Number of levels of the quantizer.
`D = 2/(L-1);`	Step size between levels (range −1 to +1).	D = 2/7 = 0.2857 for 3 bits.
`xq = round((x+1)/D)*D - 1;`	Shifts x to 0…2, counts steps, rounds to the nearest level and shifts back.	This is the quantization (rounding to the nearest level).
`e = xq - x;`	Quantization error of every sample.	Shows how much each value was changed.
`plot(t,x,t,xq,'r');`	Plots original and quantized signals together.	The staircase shape of xq is clearly seen.
`disp(max(abs(e)));`	Prints the largest error.	It must not exceed D/2 = 0.1429.

Precautions

- Use `plot` for the analog signal and `stem` for the sampled signal.
- Write `n/fs` (sample time) on the time axis when samples and the analog signal are drawn together.
- Use `.*` and `./` only for vector-by-vector operations; `2*pi*f*n/fs` is fine because f and fs are scalars.
- Do not confuse the Nyquist rate ($2f_{max}$) with the Nyquist frequency ($f_s/2$).

- Use `hold on` before drawing a second plot on the same figure.
- Do not use `L` for anything else in Program 4 after it is defined as the number of levels.

RESULTS:

Run each program in MATLAB and paste the resulting figure in the space provided below.

S.No.	Program	Output Waveform (paste figure here)
1	Sampling of a sine wave (fs = 50 Hz)	
2	Sampling at fs = 8 Hz	
3	Sampling at fs = 12 Hz	
4	Sampling at fs = 50 Hz	
5	Aliasing (9 Hz and 1 Hz)	
6	Quantization (B = 3)	

Expected Output

- **Program 1:** a smooth 5 Hz sine (5 cycles in 1 s) and 51 samples, 10 samples in each cycle.
- **Program 2:** at 8 Hz the samples do not follow the 5 Hz wave (they look like a 3 Hz wave); at 12 Hz they follow it roughly; at 50 Hz they follow it closely.
- **Program 3:** both sample lists printed in the command window are the same; the red samples lie on both the 9 Hz and the 1 Hz curves.
- **Program 4:** a staircase signal with 8 levels; step size 0.2857; maximum error about 0.143 (= Δ/2).

Table 4.4 Sampling a 5 Hz sine wave

fs (Hz)	Nyquist rate 2f (Hz)	Case	Aliasing?	Apparent frequency (Hz)	Observed
8	10	Under-sampling	Yes	$\|5 - 8\| = 3$	
12	10	Over-sampling	No	5	
50	10	Over-sampling	No	5	

Table 4.5 Quantization result (Program 4)

Bits B	Levels L	Step size Δ	Expected max. error (Δ/2)	Observed max. error
3	8	0.2857	0.1429	

Observations

Conclusion

Thus a sine wave was sampled at different sampling frequencies, aliasing was observed when the sampling frequency was less than the Nyquist rate, and the signal was quantized with 3 bits; the quantization error was verified to be within ±Δ/2 using MATLAB 7.0.

Exercise:

1. Run Program 2 with fs = 10 Hz (critical sampling). What do you see? Now change the signal to cos(2πft) and run again. Explain the difference.
2. A 7 Hz sine wave is sampled at 10 Hz. Calculate the alias frequency and verify it by changing Program 3.
3. Change B in Program 4 to 2, 4 and 8. Record the step size and the maximum error each time.

4. Find the Nyquist rate of $x(t) = \sin(2\pi \cdot 20t) + \sin(2\pi \cdot 50t)$. Sample it at 80 Hz and at 200 Hz and plot both.
5. Modify Program 1 to sample a 10 Hz cosine wave at 100 Hz.

Viva Questions:

1. What is sampling? What is the sampling period?
2. State the sampling theorem.
3. What is the difference between the Nyquist rate and the Nyquist frequency?
4. What is aliasing? How can it be avoided?
5. A 9 Hz signal is sampled at 8 Hz. What frequency will appear after sampling?
6. Why is an anti-aliasing filter placed before an ADC?
7. What is quantization? How many levels does an 8-bit ADC have?
8. What is the maximum quantization error for a step size Δ?
9. What happens to the quantization error when the number of bits increases?
10. Why do we use plot() for the analog signal and stem() for the sampled signal?

Connection to the Next Experiment:

This experiment showed how an analog signal becomes a discrete-time sequence x(n). From now on we work only with such sequences. Experiment 05 takes two sequences and combines them by **convolution** (to find the output of a system) and **correlation** (to measure how similar two signals are).